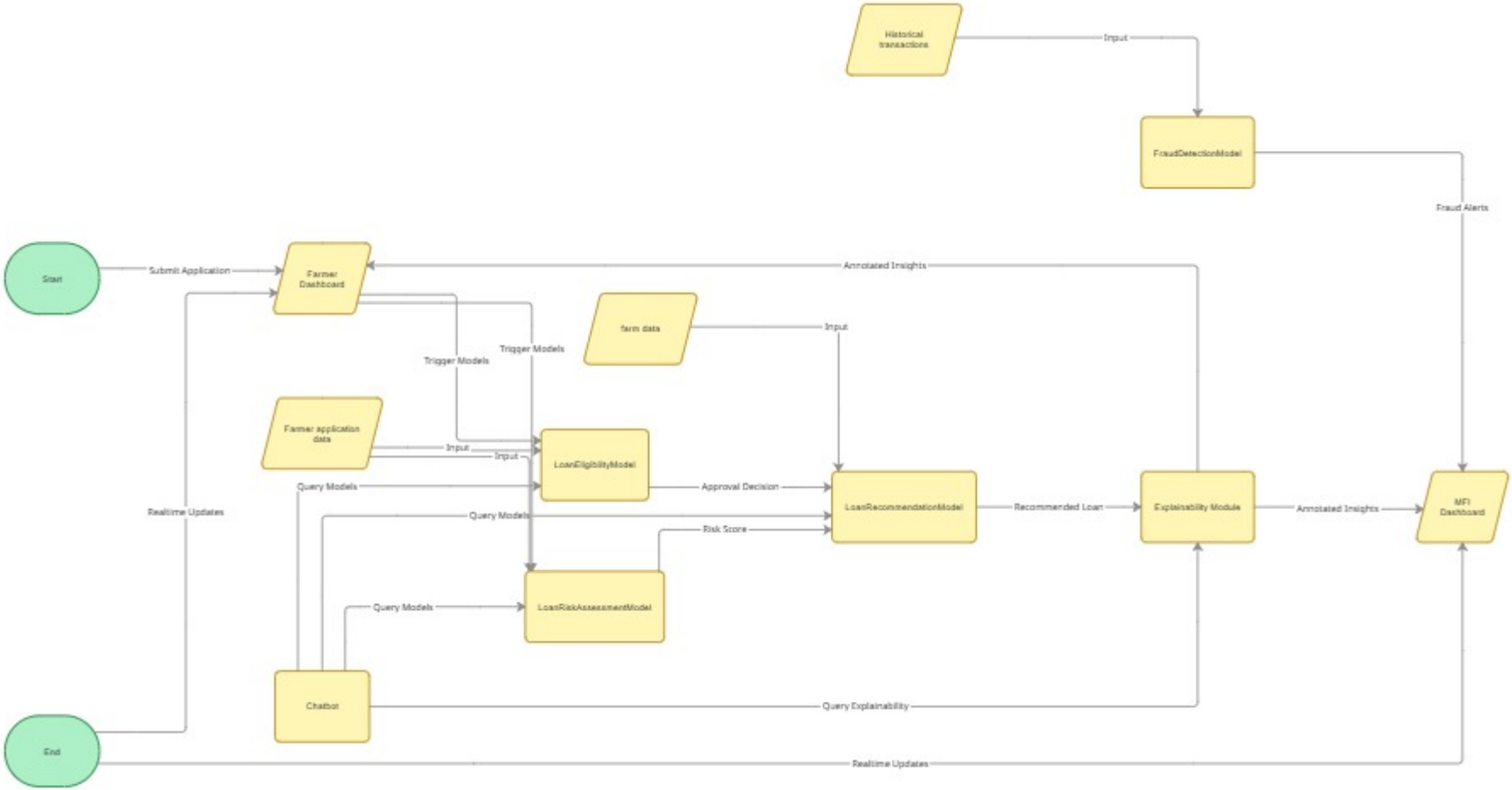
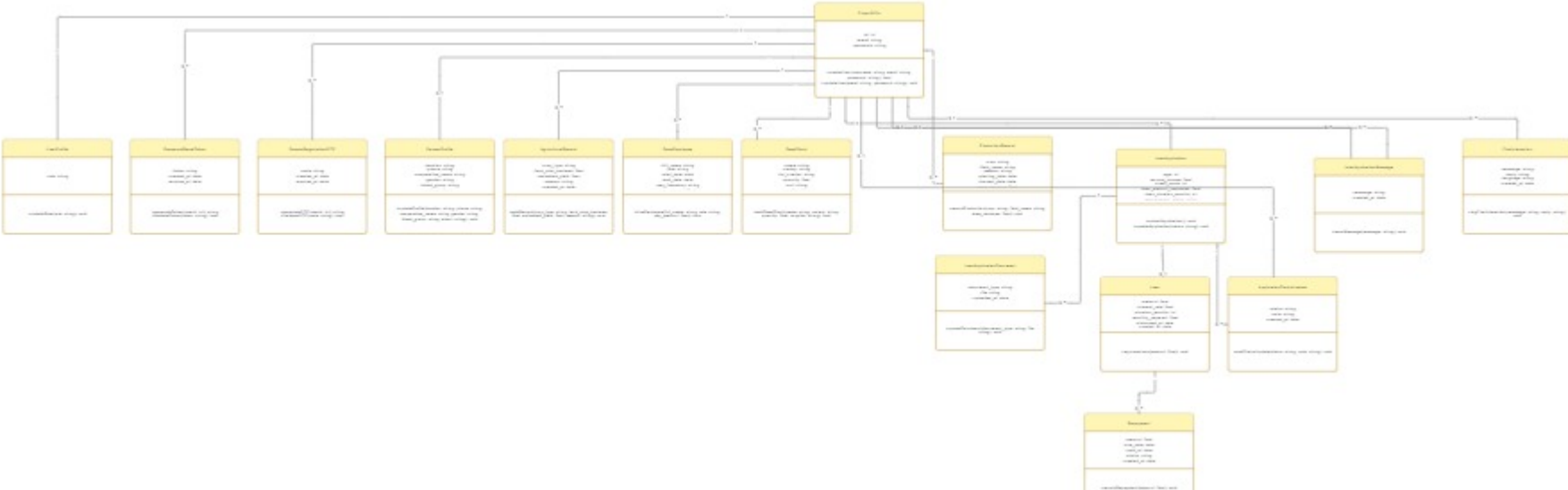
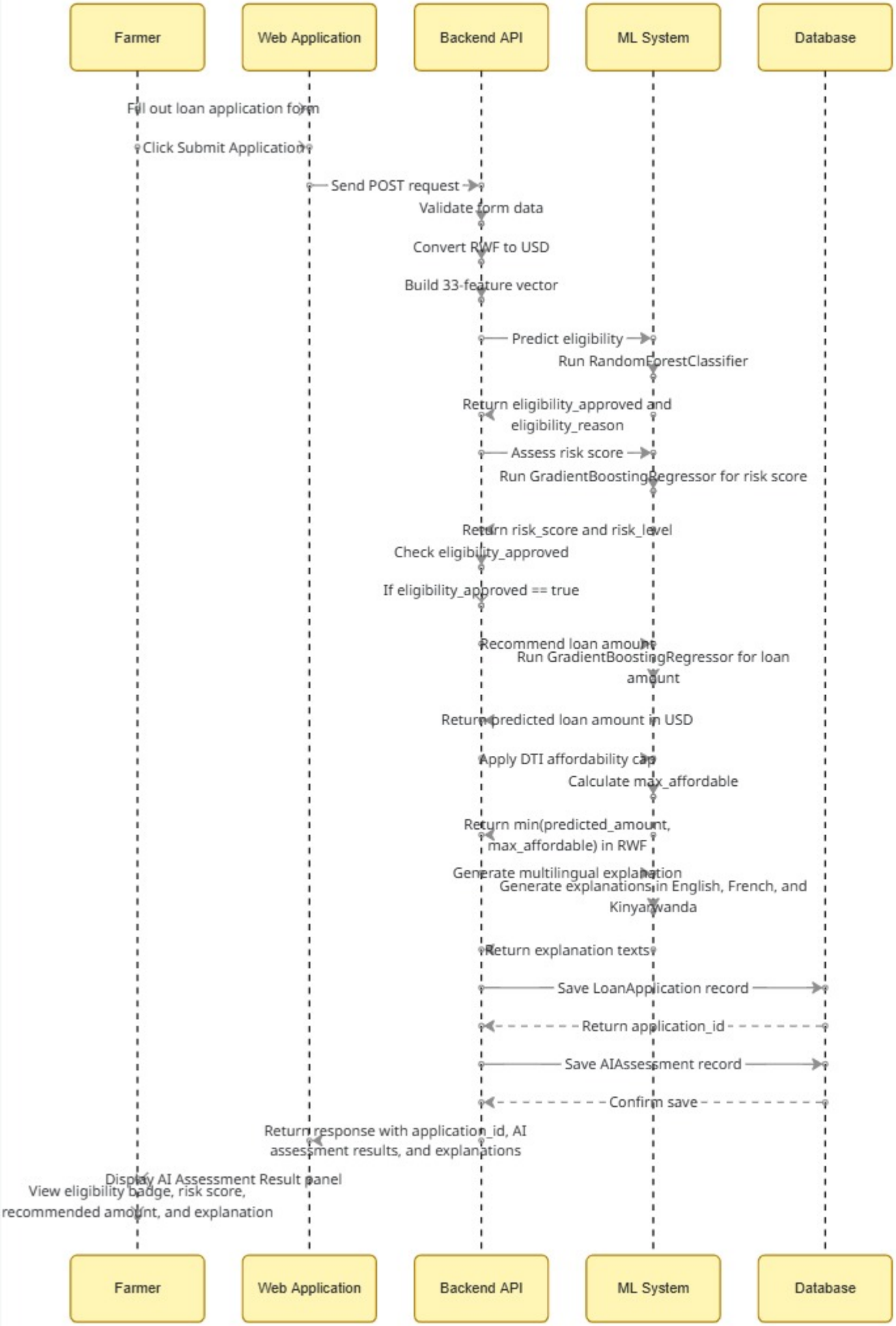
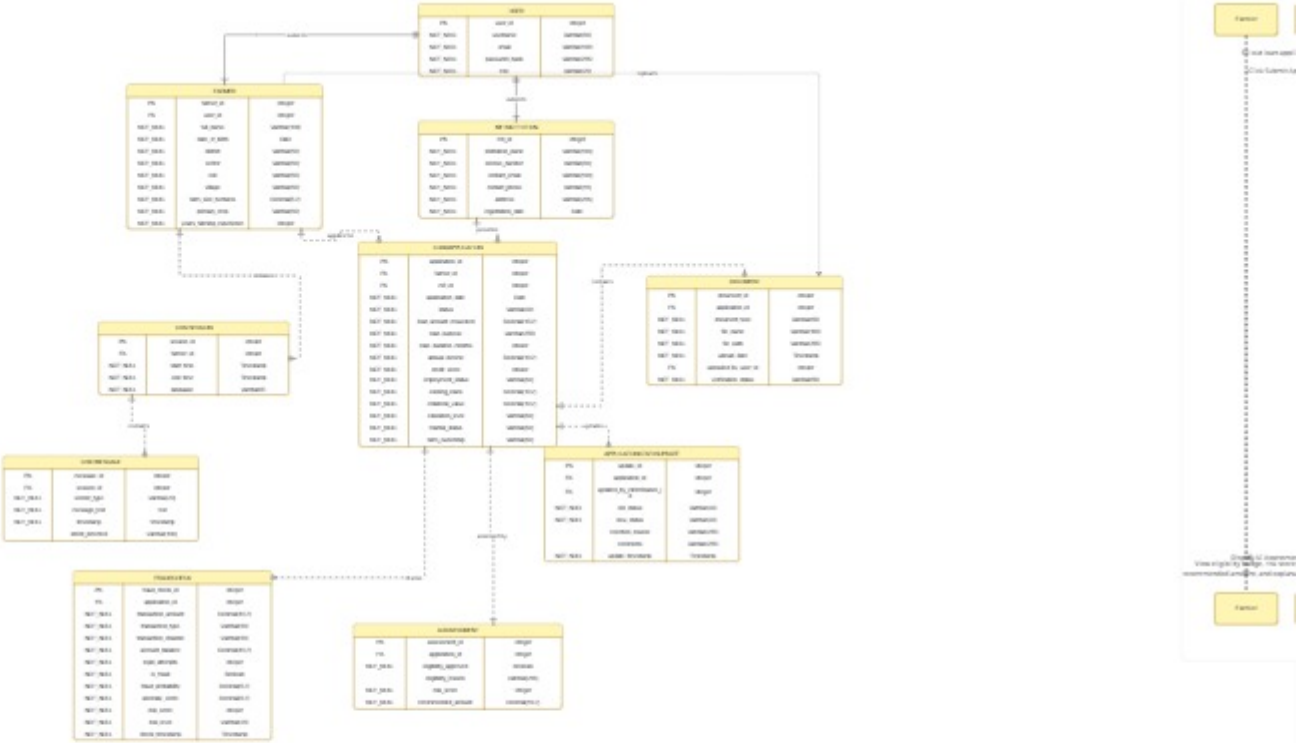
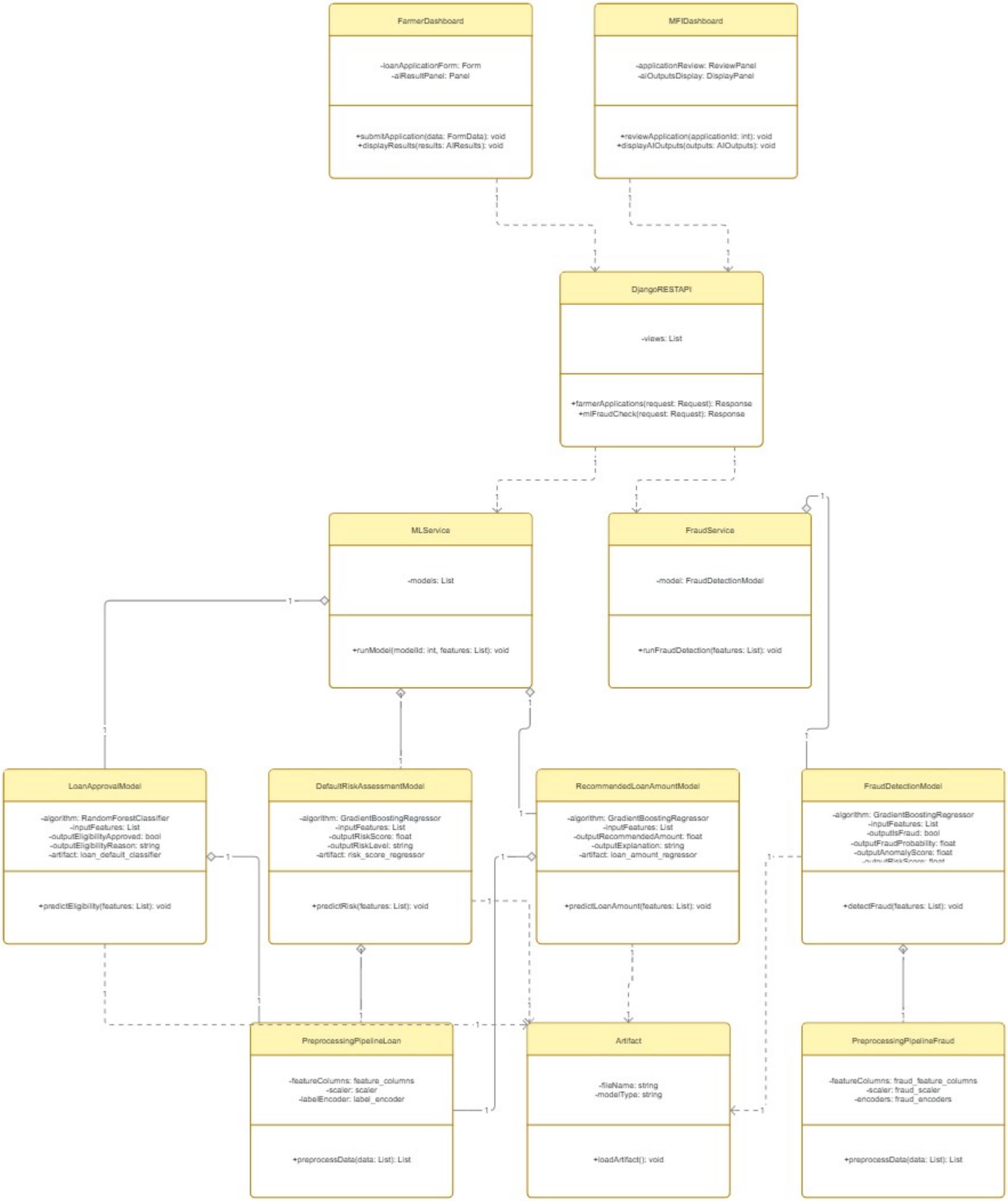

Historical transactions
Input
FraudDetectionModel
Fraud Alerts
Start
Submit Application
Farmer Dashboard
Annotated Insights
farm data
Input
Trigger Models
Trigger Models
Farmer application data
Input
Input
LoanEligibilityModel
Query Models
Approval Decision
LoanRecommendationModel
Recommended Loan
Explainability Module
Annotated Insights
MFI Dashboard
Realtime Updates
Query Models
Risk Score
Query Models
LoanRiskAssessmentModel
Chatbot
Query Explainability
End
Realtime Updates




Farmer
Web Application
Backend API
ML System
Database
Fill out loan application form
Click Submit Application
Send POST request
Validate form data
Convert RWF to USD
Build 33-feature vector
Predict eligibility
Run RandomForestClassifier
Return eligibility_approved and eligibility_reason
Assess risk score
Run GradientBoostingRegressor for risk score
Return risk_score and risk_level
Check eligibility_approved
If eligibility_approved == true
Recommend loan amount
Run GradientBoostingRegressor for loan amount
Return predicted loan amount in USD
Apply DTI affordability cap
Calculate max_affordable
Return min(predicted_amount, max_affordable) in RWF
Generate multilingual explanation
Generate explanations in English, French, and Kinyarwanda
Return explanation texts
Save LoanApplication record
Return application_id
Save AIAssessment record
Confirm save
Return response with application_id, AI assessment results, and explanations
Display AI Assessment Result panel
View eligibility badge, risk score, recommended amount, and explanation
Farmer
Web Application
Backend API
ML System
Database




FarmerDashboard
-loanApplicationForm: Form
-aiResultPanel: Panel
+submitApplication(data: FormData): void
+displayResults(results: AIResults): void
MFIDashboard
-applicationReview: ReviewPanel
-aiOutputsDisplay: DisplayPanel
+reviewApplication(applicationId: int): void
+displayAIOutputs(outputs: AIOutputs): void
DjangoRESTAPI
-views: List
+farmerApplications(request: Request): Response
+mlFraudCheck(request: Request): Response
MLService
-models: List
+runModel(modelId: int, features: List): void
FraudService
-model: FraudDetectionModel
+runFraudDetection(features: List): void
LoanApprovalModel
-algorithm: RandomForestClassifier
-inputFeatures: List
-outputEligibilityApproved: bool
-outputEligibilityReason: string
-artifact: loan_default_classifier
+predictEligibility(features: List): void
DefaultRiskAssessmentModel
-algorithm: GradientBoostingRegressor
-inputFeatures: List
-outputRiskScore: float
-outputRiskLevel: string
-artifact: risk_score_regressor
+predictRisk(features: List): void
RecommendedLoanAmountModel
-algorithm: GradientBoostingRegressor
-inputFeatures: List
-outputRecommendedAmount: float
-outputExplanation: string
-artifact: loan_amount_regressor
+predictLoanAmount(features: List): void
FraudDetectionModel
-algorithm: GradientBoostingRegressor
-inputFeatures: List
-outputIsFraud: bool
-outputFraudProbability: float
-outputAnomalyScore: float
+detectFraud(features: List): void
PreprocessingPipelineLoan
-featureColumns: feature_columns
-scaler: scaler
-labelEncoder: label_encoder
+preprocessData(data: List): List
Artifact
-fileName: string
-modelType: string
+loadArtifact(): void
PreprocessingPipelineFraud
-featureColumns: fraud_feature_columns
-scaler: fraud_scaler
-encoders: fraud_encoders
+preprocessData(data: List): List